

5 합성곱 신경망 I

5.1 합성곱 신경망

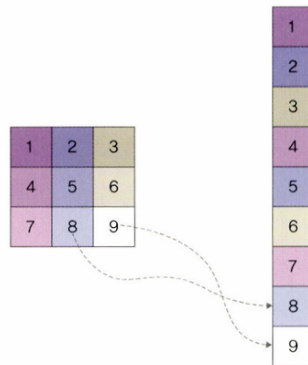
딥러닝의 역전파 과정(출력층 → 은닉층 → 입력층)은 복잡하고 많은 자원(CPU, GPU, 메모리 등)을 필요로 함. 또한 많은 계산 시간도 필요함.

⇒ 합성곱 신경망

: 이미지 전체를 한 번에 계산하는 것이 아니라 이미지의 국소적 부분을 계산함으로써 시간과 자원을 절약하여 이미지의 세밀한 부분까지 분석할 수 있도록 함.

5.1.1 합성곱층의 필요성

기존에 이미지 분석을 할 때 데이터의 공간적 구조를 무시하게 됨. 이것을 방지하고자 **합성곱층**을 도입.

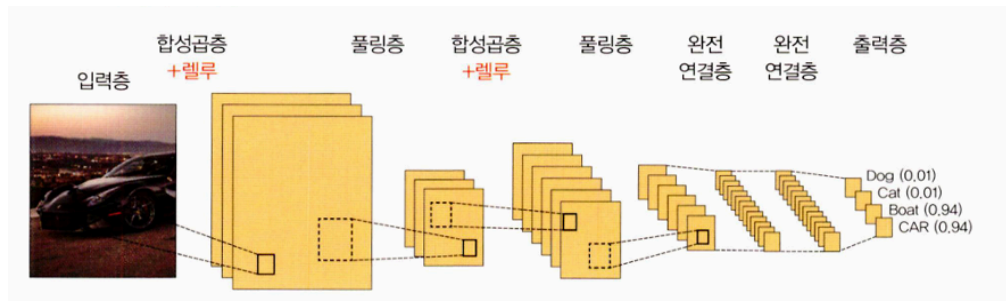


기존 이미지 분석에서는 3x3 배열을 오른쪽과 같이 펼쳐서 (flattening) 각 픽셀에 가중치를 곱해 은닉층으로 전달함.

펼치면 왜 안좋은까?

5.1.2 합성곱 신경망 구조

(1) 입력층에서 입력을 받아 (2)합성곱층과 (3)풀링층을 거치면서 입력 이미지의 주요 특성 벡터(feature vector) 를 추출. 그 후 추출된 주요 특성 벡터들은 (4)완전연결층을 거치면서 1차원 벡터로 변환되며, 마지막으로 (5)출력층에서 활성화 함수인 소프트맥스(softmax) 함수를 사용해 최종 결과를 출력



합성곱 신경망 구조

입력층(input layer)

입력 이미지 데이터가 처음으로 거치게 되는 계층.

- 이미지 데이터 : 높이(height) / 너비(width) / 채널(channel) 의 값을 갖는 3차원 데이터.
 - 채널(channel) : 이미지가 그레이스케일(흑백, gray scale) 이면 1, 컬러(RGB) 이면 3 값을 가짐
 - 이미지 형태(shape) → (4,4,3) : 높이 4 / 너비 4 / RGB 인 이미지 형식으로 표현함

합성곱층(convolutional layer)

입력 데이터에서 특성을 추출하는 역할 수

<특성 추출 과정>

- 그레이스케일 이미지
 1. 입력 이미지에 3x3 필터 적용
입력 이미지와 필터를 포개 놓고 대응되는 숫자끼리 모두 곱한 후 더함.

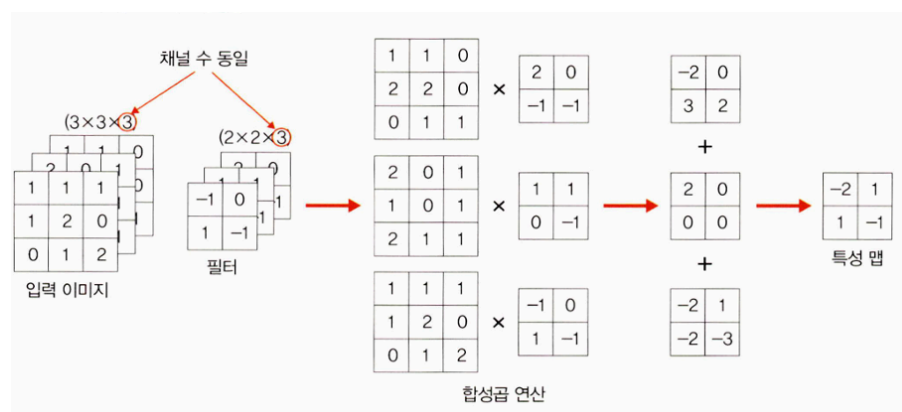


2. 스트라이드 간격에 따라 필터를 반복적으로 이동시키며 출력 배열을 채움



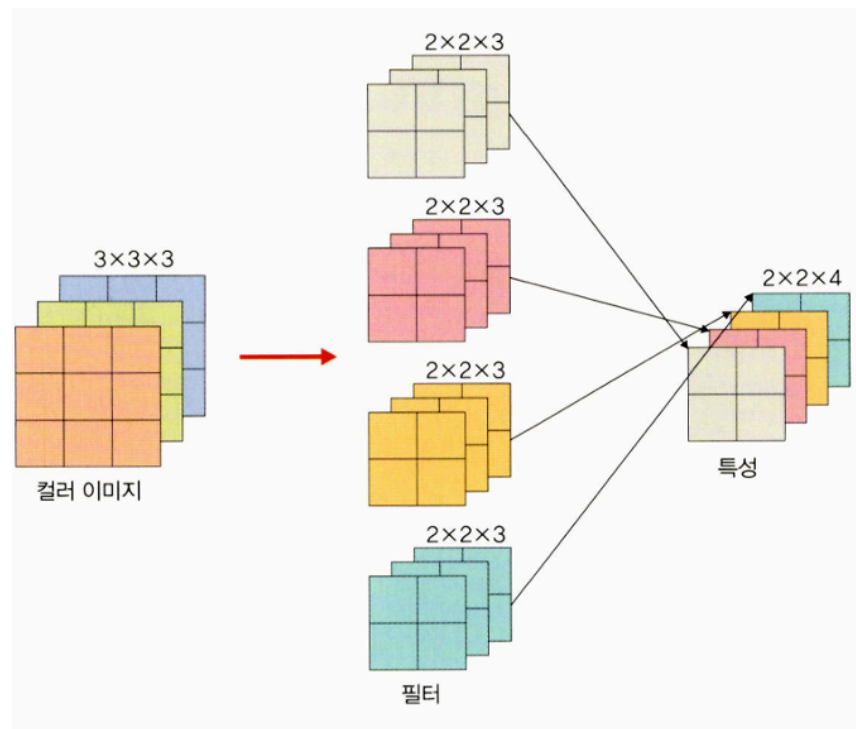
- 3×3 크기의 커널/필터가 스트라이드 1 간격으로 이동하면서 합성곱 연산을 수행함.
- (6,6,1) 원본 이미지 → (4,4,1) 특성 맵
- 컬러 이미지

필터 채널이 3 이고, RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더 해 줌.



★ 필터 **채널**이 3이지만 필터 **개수**는 1개이다.

★ 필터 **개수**가 2개 이상인 경우 필터 각각이 특성 추출 결과의 채널이 된다.



<요약>

- 입력 데이터 : (W, H, D)

- 하이퍼파라미터

 - 필터 개수 K

 - 필터 크기 F

 - 스트라이드 S

 - 패딩 P

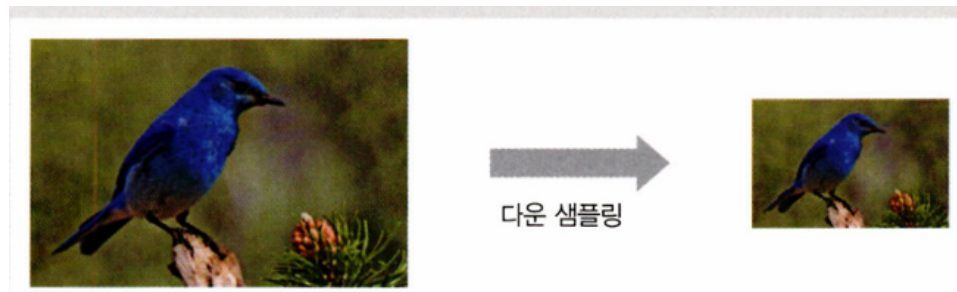
*합성곱 연산을 수행할 때 입력 데이터의 가장자리에 추가적인 픽셀을 덧붙이는 것. 합성곱 연산으로 출력 크기가 줄어드는 현상, 이로 인한 정보 손실을 방지할 수 있고, 입력과 출력 크기를 동일하게 유지해 네트워크 구조를 단순하게 만들 수 있음.

- 출력 데이터(특성 배열) : $((W-F+2P)/S+1, (H-F+2P)/S+1, K)$

풀링층(pooling layer)

합성곱층과 유사하게 특성 맵의 차원을 다운 샘플링하여 연산량을 감소시키고 주요한 특성 벡터를 추출하여 학습을 효과적으로 할 수 있게 함.

*다운 샘플링(sub-sampling) : 데이터의 해상도를 줄이거나 차원을 줄이는 과정. 리사이징, 풀링, 스트라이드 등의 방법이 있다.



리사이징을 통한 다운 샘플링

- 최대 풀링(max pooling) : 대상 영역에서 최댓값 추출
- 평균 풀링(average pooling) : 대상 영역에서 평균 반환

<최대 풀링 과정>

대상 영역에서 최댓값을 추출



스트라이드에 따라 이동하며 출력 배열을 다 채울 때까지 추출한다.

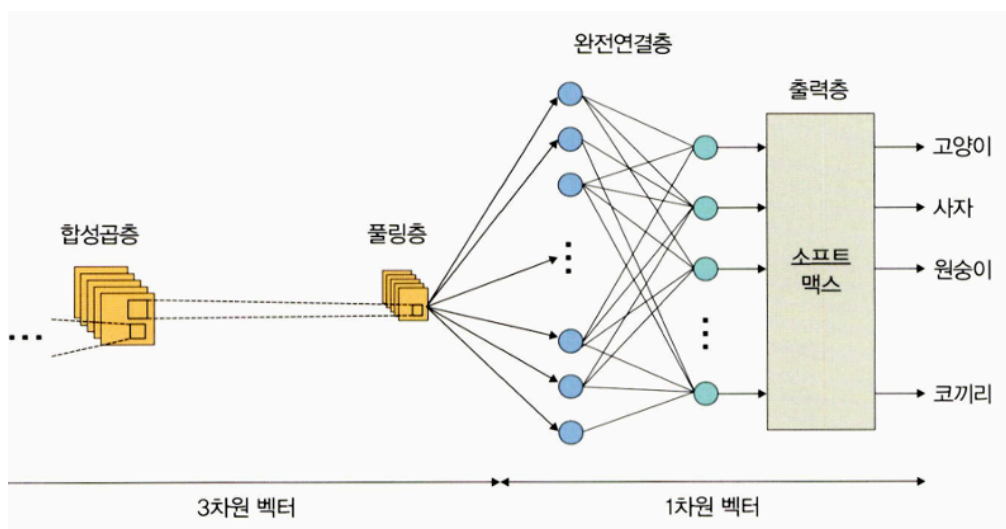


<요약>

- 입력 데이터 : (W,H,D)
- 하이퍼파라미터
 - 필터 크기 F
 - 스트라이드 S
- 출력 데이터 : ((W-F)/S+1 , (H-F)/S+1, D)

완전연결층(fully connected layer)

이미지가 3차원 벡터에서 1차원 벡터로 펼쳐지게(flatten) 됨.



출력층(output layer)

소프트맥스 활성화 함수를 사용하여 이미지가 각 레이블(label)에 속할 확률 값을 출력, 이 확률 값이 가장 높은 레이블이 최종 값으로 선정

- 소프트맥스 활성화 함수

5.1.3 1D, 2D, 3D 합성곱

1D 합성곱

필터가 시간을 축으로 **좌우로만** 이동할 수 있는 합성곱

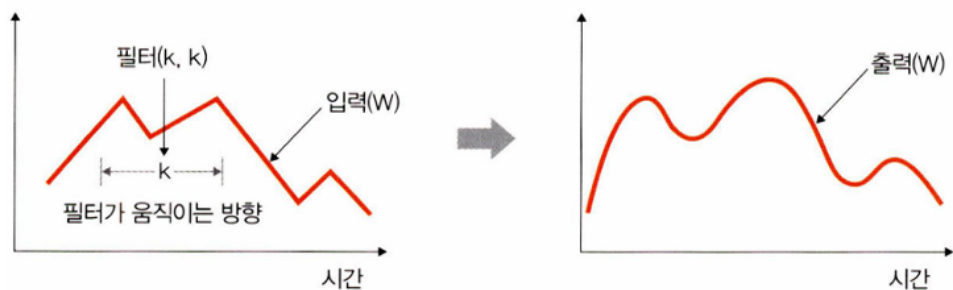
그래프의 곡선을 완화할 때 많이 사용

- 예시

입력 : [1, 1, 1, 1] (W)

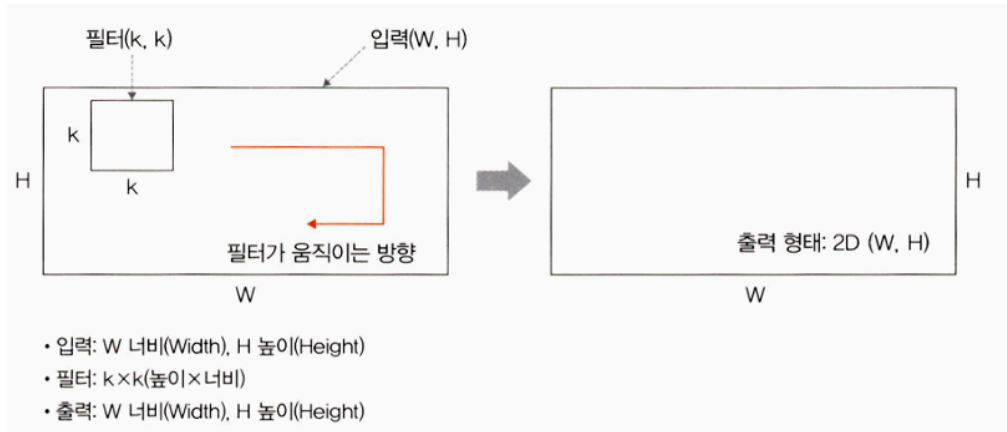
필터 : [0.25, 0.5, 0.25] (k)

출력 : [1, 1, 1] (W)



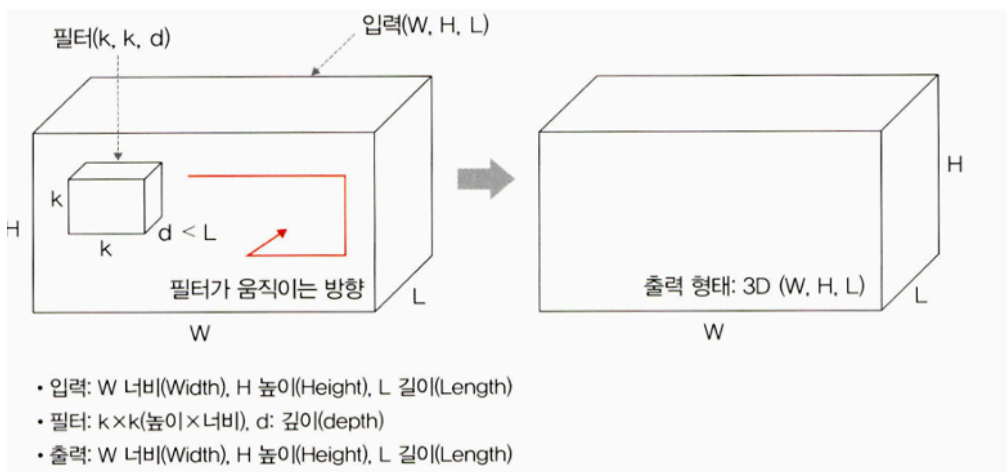
2D 합성곱

필터가 방향 두 개로 움직이는 형태



3D 합성곱

필터가 움직이는 방향이 세 개.

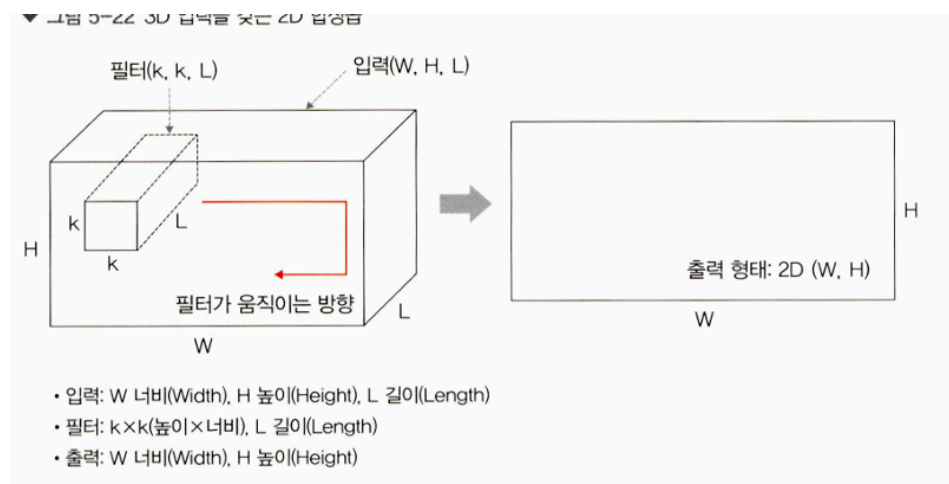


★ $d < L$ 을 유지하는 것이 중요

3D 입력을 갖는 2D 합성곱

- 입력 데이터가 **3D(예: $224 \times 224 \times 3$, $112 \times 112 \times 32$)** 형태를 갖지만, 출력은 **2D(예: $W \times H$)** 형태가 되는 합성곱 연산
- 필터(커널)의 깊이(채널 개수)는 입력 데이터의 깊이와 같아서, **입력의 전체 깊이를 한 번에 처리** 할 수 있다.
- 구조
 - 입력

- 크기: (W, H, L)
- 예: 이미지 입력이 $224 \times 224 \times 3$ (RGB 이미지) 또는 $112 \times 112 \times 32$ (CNN 중간 특징 맵)처럼 3D 형태.
- 필터 (커널)
 - 크기: (k, k, L)
 - 필터의 깊이(L)는 입력의 깊이와 동일해야 함.
- 출력
 - 크기: (W', H') → 2D 형태
 - 필터가 입력 깊이 전체를 한 번에 합성곱하기 때문에 출력은 2D가 됨.



- 대표적 사례로 LeNet-5, VGG 가 있음

1x1 합성곱

- 구조
 - 입력
 - 크기: (W, H, L)
 - 예: 이미지 입력이 $224 \times 224 \times 3$ (RGB 이미지) 또는 $112 \times 112 \times 32$ (CNN 중간 특징 맵)처럼 3D 형태.
 - 필터 (커널)
 - 크기: (1, 1, L)
 - 필터의 깊이(L)는 입력의 깊이와 동일해야 함.
 - 출력

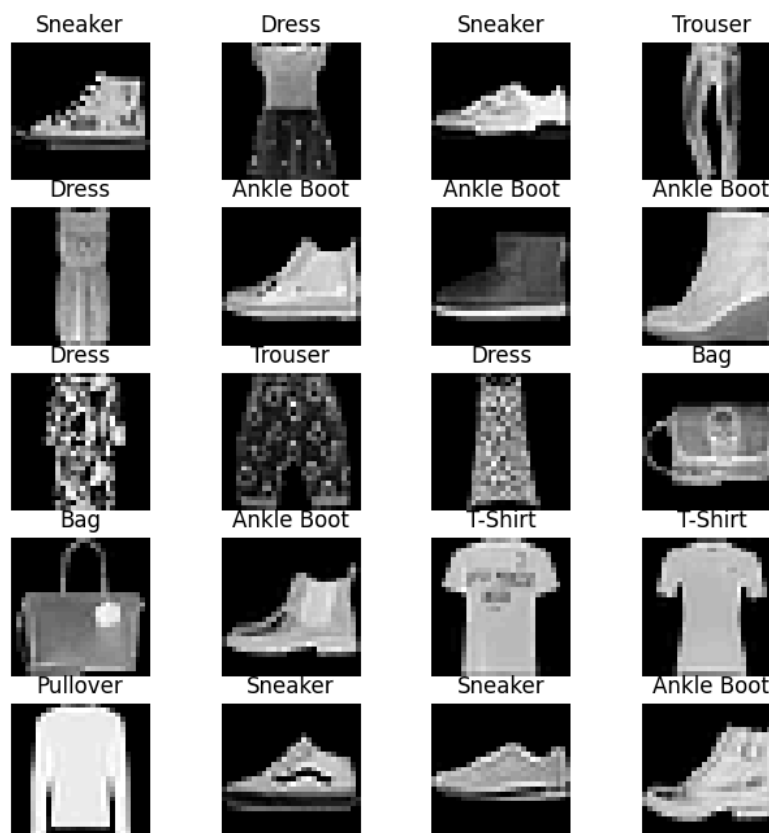
- 크기: (W, H, K)
- 1×1 합성곱에서 채널 수를 조정해서 연산량이 감소되는 효과.
- 공간적인 크기(W, H)는 유지하면서 **채널(깊이) 방향의 연산만 수행**
- 대표적 모델 - GoogLeNet

5.2 합성곱 신경망 맛보기

- 목표 : fashion_miniset 데이터셋을 사용하여 합성곱 신경망 직접 구현
 - fashion_miniset 데이터셋?

토치비전에 내장된 예제 데이터로, 운동화/셔츠/샌들 같은 작은 이미지의 모음. 열 가지로 분류할 수 있는 28x28 픽셀의 이미지 7만 개로 구성됨.

시각적으로 표현된 fashion minset 의 이미지



▼ 심층 신경망 모델 학습 (합성곱 신경망과 비교 목적)

//심층 신경망 모델 생성 (ConvNet 적용 x)

```
class FashionDNN(nn.Module):
    def __init__(self):
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(784, 256)
        self.drop = nn.Dropout(0.25)
        //p만큼의 비율로 텐서의 값이 0이 되고, 0이 되지 않는 값들은 1/(1-p)만큼 곱해서
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128,10)

    //학습 데이터를 입력받아 순전파 학습을 진행
    def forward(self, input_data):
        // veiw = 넘파이의 reshape. 2차원 텐서로 변환하되 (?,784)의 크기를 가지도록
        out = input_data.view(-1, 784)
        // 활성화 함수 relu 정의
        out = F.relu(self.fc1(out))
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

- `super(FashionDNN, self).__init__()` : FashionDNN 이라는 부모(super) 클래스를 상속받겠다는 의미
- `__init__()` : 객체가 갖는 속성 값을 초기화시키는 역할. 객체가 생성될 때 자동으로 호출된다.
- Linear : 단순 선형 회귀 모델
 - `in_features` : 입력의 크기
 - `out_features` : 출력의 크기
- nn : 딥러닝 모델 구성에 필요한 모듈이 모여 있는 패키지

 객체 지향 개념을 사용할 수 있는 이유

파이토치의 근간은 C++ 이기 때문에 C++ 에서 사용하는 객체 지향 프로그램의 특징들을 파이토치에서도 사용하게 된다.

📖 클래스와 함수

- 함수 : 하나의 특정 작업을 수행하기 위해 독립적으로 설계된 프로그램 코드.
- 클래스 : 함수를 포함한 프로그램 코드의 일부를 재사용하기 위해 함수와 관련 변수를 한번에 묶어서 재사용할 수 있게 해줌

📖 활성화 함수 정의하기

1. F.xx 또는 nn.functional.xx() : forward 함수에서 정의

```
import torch
import torch.nn as nn

inputs = torch.randn(64,3,244,244)
conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, padding=1)
#3개의 채널이 입력되어 64개의 채널이 출력되기 위해 3x3 크기의 커널 사용
#input_channel 과 output_channel 사용 연산
outputs = conv(inputs)
layer = nn.Conv2d(1,1,3)
```

2. nn.xx : __init__() 에서 정의

```
import torch.nn.functional as F

inputs = torch.randn(64,3,244,244)
weight = torch.randn(64,3,3,3)
bias = torch.randn(64)
outputs = F.conv2d(inputs, weight, bias, padding=1) #입력과 가중치 자체를 ?
```

3. 사용 방법 비교

구분	nn.xx	nn.functional.xx
형태	nn.Conv2d: 클래스 nn.Module 클래스를 상속받아 사용	nn.functional.conv2d: 함수 def function (input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	nn.Sequential 내에 위치	nn.Sequential에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

- 간단하고 직관적인 모델 $\Rightarrow$ nn.xx
- 복잡한 연산을 직접 정의하거나 커스터마이징 필요 $\Rightarrow$ nn.functional.xx (자유도 높음)

▼ 심층 신경망 파라미터 정의, 학습

#심층 신경망에서 필요한 파라미터 정의

```
learning_rate = 0.001;
model = FashionDNN();
model.to(device)
```

```
criterion = nn.CrossEntropyLoss(); #분류 문제에서 사용하는 손실 함수
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate);
print(model)
```

#심층 신경망을 이용한 모델 학습

```
num_epochs = 5
count = 5
```

```
loss_list = []
iteration_list = []
accuracy_list = []
```

```
predictions_list = []
labels_list = []
```

```
for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
```

```
    train = Variable(images.view(100, 1, 28, 28))
    labels = Variable(labels)
```

```
    outputs = model(train)
    loss = criterion(outputs, labels)
```

```

optimizer.zero_grad()
loss.backward() #Autograde 적용
optimizer.step()

loss_list.append(loss.data)
count += 1

if not (count % 50):
    total = 0
    correct = 0

    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        labels_list.append(labels)

    test = Variable(images.view(100, 1, 28, 28))

    outputs = model(test)
    predictions = torch.max(outputs, 1)[1].to(device)
    predictions_list.append(predictions)
    correct += (predictions == labels).sum()
    total += len(labels)

    accuracy = correct * 100 / total
    loss_list.append(loss.data)
    iteration_list.append(count)
    accuracy_list.append(accuracy)

if not (count % 500) :
    print("Iteration: {}, Loss: {}, Accuracy: {}%".format(count, loss.data, ac

```

- 옵티마이저를 위한 경사 하강법으로 Adam 사용

Adam 경사 하강법이란?

- Gradient 의 평균값 : 지금까지의 기울기를 누적해서 방향을 안정적으로 잡는 기능
- Gradient 제공의 평균값 : 학습 속도를 각 파라미터에 맞게 자동 조정

두 가지를 모두 고려하여 빠르고 안정적인 학습을 가능하게 함.

- Autograd 패키지 사용

Autograd : 순전파 단계에서 테이프(Tape)에 수행하는 모든 연산을 저장하고 역전파 단계에 저장된 값들을 꺼내서 사용

책에 자동 미분을 계산하기 위해 torch.autograd 패키지 안에 있는 Variable 값을 이용해야 한다고 되어 있는데 이 설명은 PyTorch 구버전 기준이라고 한다.

Pytorch 0.4 이후부터는 Variable 과 Tensor 가 완전히 통합되어

```
#이 부분을
from torch.autograd import Variable
x = Variable(torch.randn(3, 3), requires_grad=True)

#이렇게 써도 된다.
x = torch.randn(3, 3, requires_grad=True)
```

- 결과

```
➡ /usr/local/lib/python3.11/dist-packages/torch/nn/functional.py:1538: UserWarning:
  warnings.warn(warn_msg)
Iteration: 500, Loss: 0.3690766990184784, Accuracy: 83.83999633789062%
Iteration: 1000, Loss: 0.4602400064468384, Accuracy: 84.1500015258789%
Iteration: 1500, Loss: 0.2420661896467209, Accuracy: 85.63999938964844%
Iteration: 2000, Loss: 0.23568187654018402, Accuracy: 84.94000244140625%
Iteration: 2500, Loss: 0.32756656408309937, Accuracy: 85.56999969482422%
Iteration: 3000, Loss: 0.4536590874195099, Accuracy: 86.30000305175781%
```

▼ 합성곱 네트워크 생성

#합성곱 네트워크 생성

```
class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
```

```

self.layer2 = nn.Sequential(
    nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    nn.MaxPool2d(2)
)
self.fc1 = nn.Linear(in_features = 64*6*6, out_features=600)
self.drop = nn.Dropout2d(0.25)
self.fc2 = nn.Linear(in_features=600, out_features=120)
self.fc3 = nn.Linear(in_features=120, out_features=10)

def forward(self,x):
    out = self.layer1(x)
    out = self.layer2(out)
    out = out.view(out.size(0), -1)
    out = self.fc1(out)
    out = self.drop(out)
    out = self.fc2(out)
    out = self.fc3(out)
    return out

```

- **nn.Sequential()**

__init__() 함수에서 사용할 네트워크 모델들을 정의해 줌

forward() 함수에서 구현될 순전파를 계층(layer) 형태로 좀 더 가독성이 뛰어난 코드로 작성

⇒ 계층을 차례로 쌓을 수 있도록 $Wx+b$ 와 같은 수식과 활성화 함수를 연결

- **합성곱층 (conv layer)**

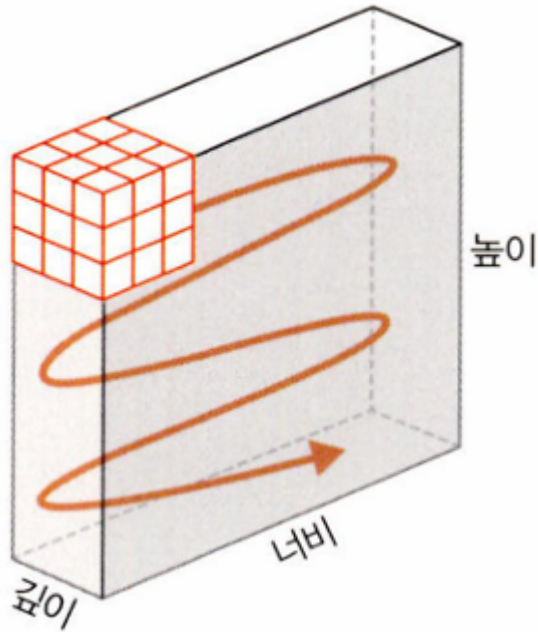
커널 이라는 $N \times M$ 크기의 행렬이 높이 $\times$ 너비 크기의 이미지를 처음부터 끝까지 훑으면서 각 원소 값끼리 곱한 후 모두 더한 값을 출력

`nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)`

(a) (b) (c) (d)

a. in_channels : 입력 채널의 수

★ 채널이란?



여기서 "깊이" 를 나타내는 것이 채널이다.

b. out_channels

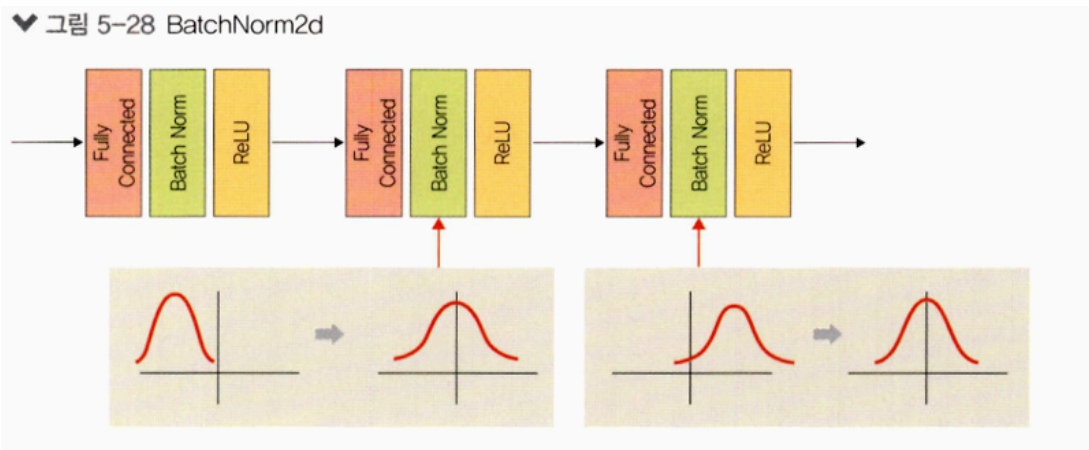
c. kernal_size : 커널 크기

★ 커널이란? 이미지 특성을 찾아내기 위한 공용 파라미터. 입력 데이터를 스트라이드 간격으로 순회하면서 합성곱을 지정

d. padding : 출력 크기를 조정하기 위해 입력 데이터 주변에 0을 채우는 것.

• BatchNorm2d

학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 가지더라도 평균과 분산을 이용하여 정규화하는 것을 의미



• MaxPool2d : 이미지 크기를 축소시키는 용도

```
nn.MaxPool2d(kernel_size=2, stride=2)
```

(a)

(b)

- a. kernal_size : 2차원 행렬로 구성된 가중치
- b. stride : 입력 데이터에 커널을 적용할 때 이동할 간격. 커질수록 출력은 작아진다
- 이미지 형태의 데이터를 분류하려면 마지막에는 완전연결층 (Fully Connected Layer) 로 보내야 함. 이때 Conv2D/MaxPool2D 등을 거쳐 출력된 feature map 을 1차원 벡터로 변환해야 함.

```
nn.Linear(in_features=64*6*6, out_features=600)
```

(a)

(b)

a. in_features : **1차원 벡터로 변환한 크기**

크기 계산법

1. Conv2D 를 사용한 경우

출력 크기 = $((W - F + 2P) / S) + 1$

- W: 입력 크기 (input size)
- F: 커널 크기 (kernel size)
- P: 패딩 (padding)
- S: 스트라이드 (stride)

2. MaxPool2D 를 사용한 경우

출력 크기 = 입력 크기 / 커널 크기

b. out_features : 출력 데이터의 크기

▼ 합성곱 네트워크 파라미터 정의 / 모델 학습 및 평가

```
#합성곱 네트워크를 위한 파라미터 정의
```

```
learning_rate = 0.001
```

```

model = FashionCNN()
model.to(device)

criterion = nn.CrossEntropyLoss();
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
print(model)

```

모델 학습 및 성능 평가

```

num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50) :
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            labels_list.append(labels)

```

```

test = Variable(images.view(100, 1, 28, 28))
outputs = model(test)
predictions = torch.max(outputs, 1)[1].to(device)
predictions_list.append(predictions)
correct += (predictions == labels).sum()
total += len(labels)

accuracy = correct * 100 / total
loss_list.append(loss.data)
iteration_list.append(count)
accuracy_list.append(accuracy)

if not (count % 500) :
    print("Iteration: {}, Loss: {}, Accuracy: {}%".format(count, loss.data, ac

```

- 결과

```

➡ Iteration: 500, Loss: 0.4751015901565552, Accuracy: 87.6500015258789%
   Iteration: 1000, Loss: 0.39670825004577637, Accuracy: 87.7699966430664%
   Iteration: 1500, Loss: 0.22866910696029663, Accuracy: 89.51000213623047%
   Iteration: 2000, Loss: 0.29824724793434143, Accuracy: 89.91999816894531%
   Iteration: 2500, Loss: 0.1493048518896103, Accuracy: 90.91999816894531%
   Iteration: 3000, Loss: 0.18844379484653473, Accuracy: 90.66999816894531%

```

합성곱 네트워크가 근소하게 더 높은 Accuracy 를 가진다.